# PL/SQL Fundamentals

## A Comprehensive Guide

Prof. A.C.S. Rao
Associate Professor

Indian Institute of Technology
Indian School of Mines

February 24, 2026

# PL/SQL

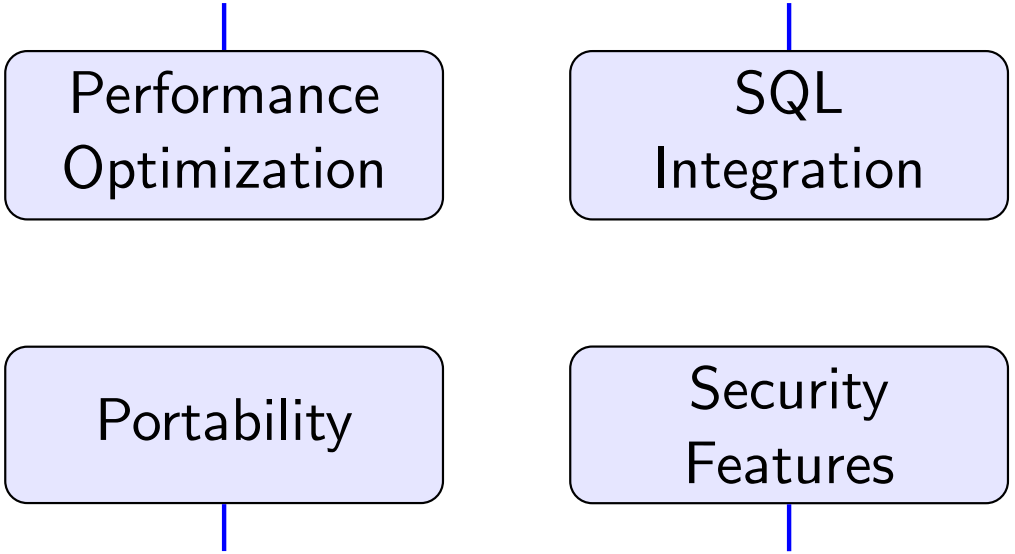
## Definition

**PL/SQL (Procedural Language/SQL)** is Oracle's procedural extension to SQL that combines database query capabilities with programming constructs.

## Core Capabilities

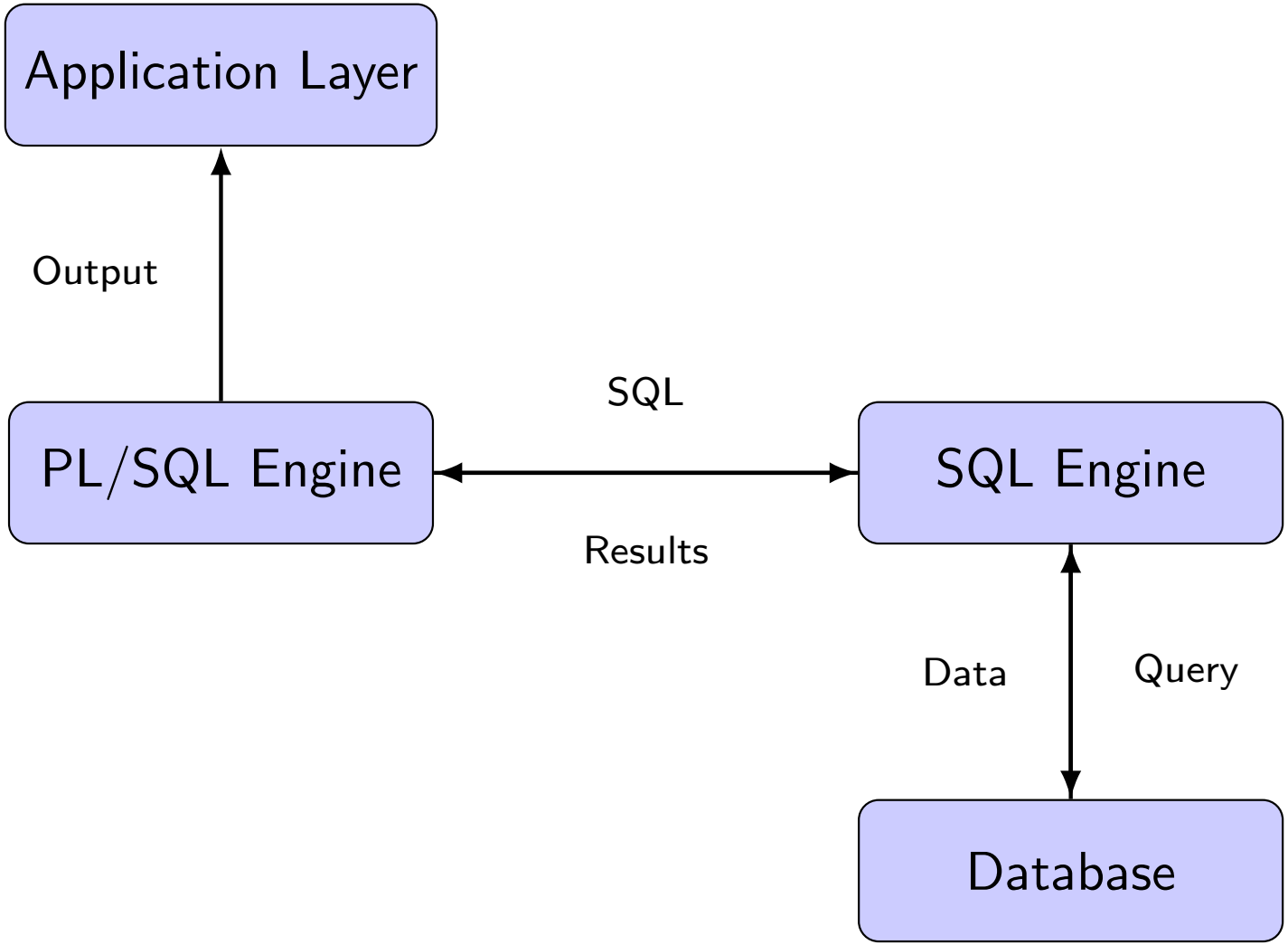
- **Procedural Logic:** IF-THEN-ELSE, loops, variables
- **SQL Integration:** Seamless embedding of DML/DDL
- **Error Management:** Structured exception handling
- **Reusability:** Stored procedures, functions, packages

# Key Advantages




- **Network Efficiency:** Single block transmission
- **Modular Design:** Encapsulated logic units
- **Maintainability:** Centralized business rules
- **Performance:** Compiled and optimized

# Architecture Overview




1. Client sends PL/SQL block
2. PL/SQL engine processes procedural code
3. SQL statements routed to SQL engine
4. Results returned through same path

# PL/SQL Block Anatomy

DECLARE (Optional)

BEGIN (Required)

EXCEPTION (Optional)

END; (Required)

```
DECLARE
    v_id NUMBER := 100;
    v_name VARCHAR2(50);
BEGIN
    SELECT last_name
    INTO v_name
    FROM employees
    WHERE employee_id = v_id;

    DBMS_OUTPUT.PUT_LINE(v_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Not found');
END;
```

# Data Type Classification

| Category | Purpose | Examples |
|---|---|---|
| Scalar | Single values | NUMBER, VARCHAR2, DATE |
| Composite | Multiple values | RECORD, TABLE, VARRAY |
| LOB | Large objects | BLOB, CLOB, BFILE |
| Reference | Pointers | REF CURSOR |

- **Scalar:** Basic data storage
- **Composite:** Structured data grouping
- **LOB:** Binary/text documents, images
- **Reference:** Dynamic result sets

# What is a PL/SQL Variable?

- A variable is a named memory location used to store data.
- In PL/SQL, variables must be declared before use.
- Declaration specifies the name and data type of the variable.

# Syntax of Variable Declaration

The basic syntax in PL/SQL:

```
variable_name data_type [NOT NULL] [:= initial_value];
```

- ‘variable$_name$‘ : *Identifierforthevariable*.

- ‘data_type‘: PL/SQL data type (e.g., NUMBER, VARCHAR2).
- ‘NOT NULL‘: Optional, variable must not be null.
- ‘:= initial_value‘: Optional initialization value.

# NOT NULL Constraint

```
DECLARE
    score NUMBER NOT NULL := 100;
BEGIN
    DBMS_OUTPUT.PUT_LINE(score);
END;
```

The ‘NOT NULL‘ constraint prevents assignment of NULL.

# Data Type Examples

Common data types used while declaring variables:

- NUMBER – Numeric values
- VARCHAR2 – Text
- BOOLEAN – TRUE/FALSE
- DATE – Date and time

# Summary

- Variables must be declared in the ‘DECLARE‘ section.
- Initialization is optional but useful.
- Data type and constraints determine valid values.

# What is Decision Making?

- Decision making allows PL/SQL to execute specific statements based on conditions.
- ‘IF...THEN‘, ‘ELSE‘, ‘ELSIF‘, and nested ‘IF‘ are used for control flow.

# IF...THEN Statement

```
IF condition THEN
   statements;
END IF;
```

If the condition evaluates to TRUE, the statements inside are executed.

## Example

```
DECLARE
   num NUMBER := 10;
BEGIN
   IF num > 5 THEN
      DBMS_OUTPUT.PUT_LINE('Greater than 5');
   END IF;
END;
```

# IF...THEN...ELSE

```
IF condition THEN
   statements;
ELSE
   other_statements;
END IF;
```

Executes the ‘ELSE‘ block if the condition is FALSE.

## Example

```
DECLARE
   num NUMBER := 3;
BEGIN
   IF num > 5 THEN
      DBMS_OUTPUT.PUT_LINE('Greater than 5');
   ELSE
      DBMS_OUTPUT.PUT_LINE('5 or less');
   END IF;
END;
```

# IF...ELSIF...ELSE

```
IF condition1 THEN
   statements;
ELSIF condition2 THEN
   other_statements;
ELSE
   default_statements;
END IF;
```

Allows checking multiple conditions in sequence.

## Example

```
DECLARE
   num NUMBER := 8;
BEGIN
   IF num < 5 THEN
      DBMS_OUTPUT.PUT_LINE('Less than 5');
   ELSIF num = 5 THEN
      DBMS_OUTPUT.PUT_LINE('Equal to 5');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Greater than 5');
   END IF;
END;
```

# Nested IF

- You can place an ‘IF...THEN‘ inside another ‘IF...THEN‘ block.
- Useful when decisions are dependent on previous conditions.

## Example

```
DECLARE
   x NUMBER := 10;
BEGIN
   IF x > 5 THEN
      IF x < 15 THEN
         DBMS_OUTPUT.PUT_LINE('Between 5 and 15');
      END IF;
   END IF;
END;
```

# Summary

- Decision making in PL/SQL uses conditional control structures.
- ‘IF...THEN‘, ‘IF...ELSE‘, ‘ELSIF‘, and nested ‘IF‘ allow flexible logic.
- Helps execute statements based on dynamic conditions.

# Introduction to Loops

- Loops are used to repeatedly execute a block of statements.
- PL/SQL supports three types of loops:
  - Basic **LOOP**
  - **WHILE** loop
  - **FOR** loop

# Basic LOOP Structure

```
LOOP
    statements;
END LOOP;
```

Must use EXIT to terminate the loop.

## Example

```
DECLARE
    i NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        EXIT WHEN i = 5;
        i := i + 1;
    END LOOP;
END;
```

# WHILE LOOP Structure

```
WHILE condition LOOP
    statements;
END LOOP;
```

Executes while the condition is TRUE.

## Example

```
DECLARE
    i NUMBER := 1;
BEGIN
    WHILE i <= 5 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        i := i + 1;
    END LOOP;
END;
```

# FOR LOOP

```
FOR i IN 1..5 LOOP
    statements;
END LOOP;
```

Automatically handles loop counter.

## Example

```
BEGIN
    FOR i IN 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;
END;
```

# Summary

- PL/SQL provides basic, WHILE, and FOR loops.
- ‘EXIT WHEN‘ is used to break out of loops.
- Loops help repeat operations efficiently.

# What is SELECT INTO?

- ‘SELECT INTO‘ fetches query results directly into PL/SQL variables.
- It retrieves values from a table and stores them in declared variables.
- Typically used when exactly one row is expected.

# Syntax

```
SELECT col1, col2, ...
INTO var1, var2, ...
FROM table_name
WHERE condition;
```

- Number of selected columns must match number of variables.
- Variable types should be compatible with column types.

# Example: Fetch Single Value

```
DECLARE
    v_salary NUMBER;
BEGIN
    SELECT SALARY INTO v_salary
    FROM EMPLOYEE
    WHERE EMP_ID = 1;

    DBMS_OUTPUT.PUT_LINE(v_salary);
END;
```

- Prints the salary of employee with ‘EMP$_ID = 1$‘.

# Example: Multiple Columns

```
DECLARE
    v_sal   NUMBER;
    v_age   NUMBER;
BEGIN
    SELECT SALARY, AGE
    INTO v_sal, v_age
    FROM EMPLOYEE
    WHERE EMP_ID = 1;

    DBMS_OUTPUT.PUT_LINE(v_sal || ', ' || v_age);
END;
```

- Fetches both salary and age into variables.

# Important Points

- ‘SELECT INTO‘ returns exactly one row — otherwise exceptions occur.
- Use exception handlers for:
  - ‘NO_DATA_FOUND‘ when no row matches.
  - ‘TOO_MANY_ROWS‘ when more than one row is returned.
- For multiple rows, use cursors instead.
- ‘SELECT INTO‘ is a quick way to fetch single-row query results into PL/SQL variables.
- Always match column and variable counts and compatible types.
- Handle exceptions to avoid runtime errors.

# What is the INSERT Statement?

- The ‘INSERT‘ statement adds new records to a database table.
- It is a key part of Data Manipulation in PL/SQL.
- Used to populate tables with required data.

# Syntax: INSERT with Columns

```sql
INSERT INTO table_name
  (column1, column2, ...)
VALUES
  (value1, value2, ...);
```

- Specify the table and target columns.
- Values must match order and type of columns.

## Syntax: Insert Without Column List

```sql
INSERT INTO table_name
VALUES
  (value1, value2, ...);
```

- Inserts values into all columns in defined order.

# Example: Insert a Single Row

```
BEGIN
   INSERT INTO employees
     (employee_id, first_name, last_name, department)
   VALUES (101, 'John', 'Doe', 'Sales');
END;
```

- Adds a new employee record.

## Example: Insert Multiple Rows

```
BEGIN
   INSERT INTO employees
     (employee_id, first_name, last_name, department)
   VALUES
     (102, 'Alice', 'Smith', 'Marketing'),
     (103, 'Bob', 'Johnson', 'Finance'),
     (104, 'Emily', 'Brown', 'HR');
END;
```

- Inserts several records in one statement.

# Insert with a SELECT Query

```
BEGIN
   INSERT INTO new_employees
   SELECT * FROM employees
   WHERE department = 'Sales';
END;
```

- Copies matching rows from one table to another.

# Important Notes

- The number of values must match the number of columns.
- Ensure correct data types for inserted values.
- 'INSERT INTO ... SELECT' helps populate tables from existing data.
- 'INSERT' adds new rows to tables in PL/SQL.
- You can insert single rows, multiple rows, or use a SELECT query to insert.
- Essential for managing persistent data.

# What is the UPDATE Statement?

- The ‘UPDATE‘ statement is used to modify existing data in a table.
- You can change one or more columns in one or more rows.
- The scope of the update is controlled using the ‘WHERE‘ clause.

## Syntax

```
UPDATE table_name
SET column1 = value1,
    column2 = value2,
    ...
WHERE condition;
```

- ‘table_name‘: The target table to update.
- ‘SET‘: Assigns new values to one or more columns.
- ‘WHERE‘: Specifies which rows are affected (optional but recommended).

# Example: Update a Column

```
BEGIN
  UPDATE employee
  SET salary = 70000
  WHERE employee_id = 101;
END;
```

- Updates salary for employee with ‘employee$_id = 101$‘.

## Example: Update Multiple Columns

```
BEGIN
  UPDATE employee
  SET department = 'HR',
      salary = salary + 5000
  WHERE department = 'Sales';
END;
```

- Modifies department and increases salary for matching rows.

# Danger of Missing WHERE

- If the ‘WHERE‘ clause is omitted, all rows in the table are updated.
- This is often unintended and affects data integrity.
- Always use a filter to control which rows should change.

# Important Notes

- Ensure column types and values are compatible.
- Use ‘COMMIT‘ after update in PL/SQL to make changes permanent.
- Combined with transactions and exception handling for safety.
- ‘UPDATE‘ modifies existing table data.
- Use ‘SET‘ for new column values.
- Always use ‘WHERE‘ to limit affected rows unless intentional.

# What is the DELETE Statement?

- The ‘DELETE‘ statement removes existing rows from a table.
- Part of Data Manipulation Language (DML).
- Control which rows to delete using a ‘WHERE‘ clause.

## DELETE Syntax

```
DELETE FROM table_name
WHERE condition;
```

- ‘table_name‘: Target table to delete rows from.
- ‘WHERE‘: Filters the rows to remove.

# Example: Delete Matching Rows

```
BEGIN
    DELETE FROM employees
    WHERE department = 'Sales';
END;
```

- Removes all rows where ‘department = ’Sales’‘.

## Example: Delete All Rows

```
BEGIN
    DELETE FROM employees;
END;
```

- Deletes every row in the ‘employees‘ table.
- Equivalent to ‘TRUNCATE‘ but logs each row deletion.

# Important Points

- Without a ‘WHERE‘ clause, **all rows will be removed**.
- Use filters carefully to avoid accidental data loss.
- Use ‘COMMIT‘ to make changes permanent (if in transactional mode).
- Use ‘WHEN OTHERS THEN‘ to catch unexpected errors.
- Rollback in exception handlers to restore state if needed.
- ‘DELETE‘ removes unwanted rows.
- Always use ‘WHERE‘ unless you intend to clear the whole table.
- PL/SQL wraps DML inside ‘BEGIN–END‘ blocks.

# PL/SQL — ‘ORDER BY‘ Clause

- Used to sort the result set of a query in ascending or descending order.
- Often used with ‘SELECT‘ inside PL/SQL blocks.

**Syntax:**

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1 [ASC|DESC];
```

**Example (Ascending Order):**

```
BEGIN
   SELECT emp_name, salary
   INTO v_name, v_sal
   FROM employees
   ORDER BY salary ASC;
END;
```

# PL/SQL — 'GROUP BY' Clause

- Used to group rows that have the same values in specified columns.
- Often combined with aggregate functions.

**Syntax:**

```
SELECT column1, AGG_FUNC(column2)
FROM table_name
GROUP BY column1;
```

**Example:**

```
BEGIN
   SELECT department, COUNT(*)
   INTO v_dept, v_count
   FROM employees
   GROUP BY department;
END;
```

# Key Points — ORDER BY vs GROUP BY

- ‘ORDER BY‘ → sorts result sets.
- ‘GROUP BY‘ → aggregates rows into groups.
- ‘GROUP BY‘ requires all non-aggregated columns to be listed.
- ‘ORDER BY‘ appears at the end of the query.
- Both can be used together (sort grouped results).

**Example with GROUP BY and ORDER BY**

```sql
SELECT dept, COUNT(*)
FROM employees
GROUP BY dept
ORDER BY COUNT(*) DESC;
```

# Introduction to PL/SQL Operators

- Operators are symbols or keywords used to perform comparisons and logical operations.
- Used in ‘WHERE‘ clauses and control structures (‘IF‘, ‘LOOP‘, etc.).
- **Comparison Operators:** $=, <>, >, <, >=, <=$
- **Logical Operators:** AND, OR, NOT
- **Pattern matching:** LIKE
- **NULL handling:** IS NULL, IS NOT NULL

# Logical Operators: AND / OR

- **AND:** True if **all** conditions are true.

```sql
SELECT * FROM emp
WHERE dept = 'Sales'
  AND salary > 50000;
```

- **OR:** True if **any** condition is true.

```sql
SELECT * FROM emp
WHERE dept = 'Sales'
  OR dept = 'HR';
```

# Logical Operator: NOT

- **NOT** reverses the truth of a condition.

```
SELECT * FROM emp
WHERE NOT (salary > 50000);
```

- True when the condition inside is FALSE.

# Pattern Matching: LIKE

- **LIKE** is used to compare strings using wildcards.
- Wildcards:
  - '%' → zero or more characters
  - '_' → exactly one character

```
SELECT * FROM emp
WHERE name LIKE 'A%';
```

- Returns names starting with 'A'.

# NULL Handling: IS NULL / IS NOT NULL

- **IS NULL** checks for NULL values.

```
SELECT * FROM emp
WHERE address IS NULL;
```

- 'IS NOT NULL' checks for non-NULL values.

```
SELECT * FROM emp
WHERE address IS NOT NULL;
```

# PL/SQL Triggers

- A trigger is a **PL/SQL block automatically executed** when a specific event occurs on a table or view.
- Common events: ‘INSERT‘, ‘UPDATE‘, ‘DELETE‘.
- Trigger timing can be:
  - ‘BEFORE‘ — runs before the DML action
  - ‘AFTER‘ — runs after the DML action
- Triggers can be row-level (‘FOR EACH ROW‘) or statement-level.

**Trigger Example:**

```
CREATE OR REPLACE TRIGGER trg_log_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('Insert detected');
END;
```

# Understanding PL/SQL Triggers

## Definition and Core Concepts

### Definition

A **database trigger** is a stored PL/SQL block that automatically executes (fires) in response to specific events occurring on database tables or views.

### Key Characteristics:

- **Automatic Execution:** Fires implicitly when triggering event occurs
- **Event-Driven:** Responds to DML operations (INSERT, UPDATE, DELETE)
- **Transparent:** Executes without user intervention
- **Atomic:** Part of the transaction context

### Common Applications:

- Data validation and cleansing
- Audit logging and change tracking
- Business rule enforcement
- Derived column calculations

# Trigger Events & Classification Matrix

| Event Type | DML Operations | System Events |
|---|---|---|
| **Data Changes** | INSERT<br>UPDATE<br>DELETE | Server startup/shutdown<br>Logon/Logoff<br>DDL operations |
| **Timing** | BEFORE / AFTER | INSTEAD OF |
| **Level** | Row-level (FOR EACH ROW) | Statement-level |

Table: Trigger classification framework

**Event Characteristics:**

- **BEFORE Triggers:** Pre-validation, default value assignment
- **AFTER Triggers:** Post-operation auditing, cascade updates
- **Row-Level:** Fires for each affected row
- **Statement-Level:** Fires once regardless of rows affected

# Trigger Architecture & Syntax Structure

## Trigger Syntax Anatomy

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF}
{INSERT | UPDATE | DELETE}
ON table_name
[FOR EACH ROW]
[WHEN (condition)]
[FOLLOWS another_trigger]
[DECLARE
   -- Variable declarations]
BEGIN
   -- PL/SQL execution logic
   -- Access :NEW and :OLD pseudo-records
EXCEPTION
   -- Exception handling
END;
/
```

**Key Components:**

- **Trigger Name:** Unique within schema, descriptive naming convention
- **Timing Point:** Determines execution phase relative to DML
- **Event Clause:** Specifies triggering DML operations

# Practical Implementation Examples

## Example 1: Audit Logging Trigger

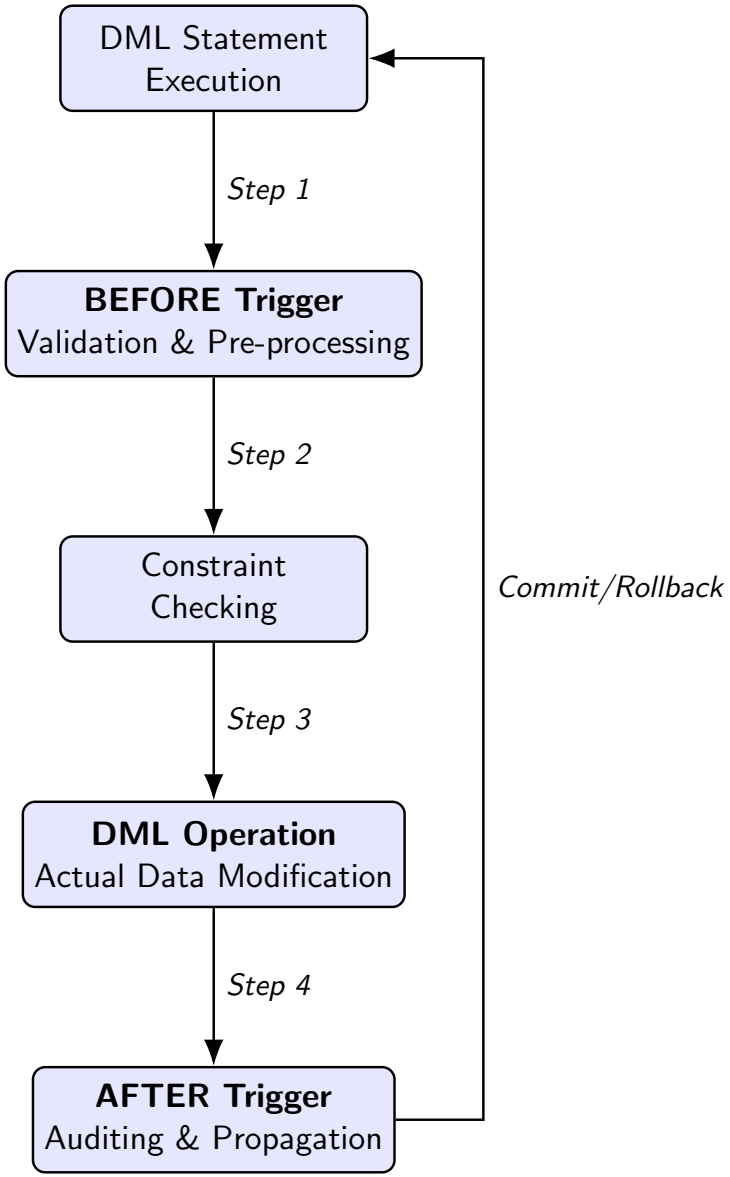
```
CREATE OR REPLACE TRIGGER trg_employee_audit
AFTER INSERT OR UPDATE OR DELETE ON employees
FOR EACH ROW
DECLARE
   v_action VARCHAR2(10);
BEGIN
   v_action := CASE
      WHEN INSERTING THEN 'INSERT'
      WHEN UPDATING THEN 'UPDATE'
      WHEN DELETING THEN 'DELETE'
   END;

   INSERT INTO employee_audit_log (
      employee_id, action, old_salary,
      new_salary, changed_by, change_date
   ) VALUES (
      NVL(:NEW.employee_id, :OLD.employee_id),
      v_action,
      :OLD.salary,
      :NEW.salary,
      USER,
      SYSDATE
   );
END;
/
```

**Design Guidelines:**

- Keep trigger logic concise and focused
- Avoid recursive trigger chains
- Use WHEN clause for conditional firing
- Document trigger dependencies
- Test thoroughly with bulk operations
- Consider compound triggers for complex scenarios

**Performance Considerations:**

- Row-level triggers impact bulk operations
- Minimize database calls within triggers
- Use PRAGMA AUTONOMOUS_TRANSACTION cautiously

# Trigger Execution Flow & Monitoring